paper or electronic form. You get yourself signed in and are given a badge to wear around the convention halls to save you from having to reidentify yourself again and again. The badge will include your information encoded as a barcode or a QR code, so those who need to know your identity can use a decoder (scanner) to access it quickly without having to ask you for it. The badge will expire at some point—perhaps at the end of the day or convention.

Authentication in a web application follows a similar process. A Razor Pages application visitor is challenged to provide their identification by completing some kind of sign-in form. Once signed in, authentication services serialize information about the user into an encrypted cookie, which is the equivalent to the badge that you wear around a convention. The cookie is passed between the client and server for all subsequent requests, until the cookie expires. Authentication middleware reads values from this cookie and uses them to hydrate the `User` property of the `HttpContext`. Thereafter, any service that needs information about the current user can check the `User` property, which is a `ClaimsPrincipal` type. The equivalent to a trade convention visitor is represented within a Razor Pages application by this instance of the `ClaimsPrincipal`.

9.1.2 Adding simple authentication

This section shows the minimum steps required to add cookie-based authentication to the CityBreaks application. You will add authentication services to the service container, specifying some default information about how the authentication service should work. Then you will add authentication middleware enabling authentication capabilities. Finally, you will add a simple login form to obtain the user's credentials and authenticate them.

First you need to add a using directive to Program.cs to make `Microsoft.AspNetCore .Authentication.Cookies`